Geo-Distributed Hybrid Retrieval-Augmented Generation: Placement Tradeoffs and Speculative Prefill Acceleration

Anonymous Author(s)

ABSTRACT

Retrieval-Augmented Generation (RAG) systems are increasingly deployed across geographically distributed, heterogeneous nodes, yet the placement of their individual pipeline stages—sparse retrieval, dense retrieval, fusion, hydration, and generation—remains an open systems question. We present a cache-state-aware measurement study of stage-to-node placement for a *hybrid* RAG pipeline (BM25 + BGE-M3 dense retrieval with reciprocal-rank fusion, and a vLLM-served Qwen3.8-27B generator) on commodity hardware: an RTX 3090 generation host and a GTX 1660 Ti laptop gateway, connected over a WireGuard link shaped by *bidirectional* Linux `netem` across four WAN regimes (0/15/40/80 ms), with 120 seeded queries per arm, three repeats, and cold-prefix and warm cache states reported separately. Three findings reframe placement for this class of systems. First, TTFT is essentially *flat* across the swept RTT range (a measured sensitivity of ≈0.95 ms of TTFT per ms of RTT in the warm state): an identifier-sized placement boundary makes the network regime a second-order concern. Second, KV-cache state, not the network, dominates latency variance—cold-prefix TTFT (≈1.7 s) is 2.8–3.2× warm TTFT (≈0.51–0.59 s). Third, **SPHP** (Speculative Progressive Hydration & Prefill), which prefills the generator on a provisional sparse hint while the dense leg is still in flight, cuts cold-prefix TTFT by 23.7%–27.6% at all four RTT tiers (one-sided permutation tests, Holm-FWER, all $p \leq 0.0008$) with a measured hit rate of $h = 0.900$ (95% CI $[0.883, 0.914]$), while warm-state savings are within noise at the larger sample (0 of 8 warm cells significant; the earlier N=50 warm significance was a small-sample artifact whose point estimates fell from -9.7% to -4.3%); an answer-level comparison bounds the hit-path quality cost (token-F1 0.42 versus a 0.59 resampling-noise ceiling). We further derive an analytical placement cost model validated at the regime level (leave-one-tier-out MAPE 2.2% cold, 4.1% warm for P2) and use it to show that the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space. All artifacts, including the measurement harness and scoring pipeline, are released.

ACM Reference Format:
Anonymous Author(s). 2026. Geo-Distributed Hybrid Retrieval-Augmented Generation: Placement Tradeoffs and Speculative Prefill Acceleration. In . ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) has become the dominant architecture for grounding large language models (LLMs) in external knowledge [5, 13, 15, 24]. A modern RAG pipeline is not a single monolith but a sequence of heterogeneous stages: a *sparse* lexical search (e.g., BM25 [34]), a *dense* semantic search over learned embeddings [8, 19, 33], a *fusion* step that merges the two ranked lists (e.g., reciprocal-rank fusion [10]), a *hydration* step that fetches the full text of the selected passages, and finally *generation*, in which an LLM [31, 38] is prefilled with the retrieved context and autoregressively decodes an answer [22].

As these systems move from a single datacenter box to the edge, a fundamental *placement dilemma* emerges. The stages have wildly different resource profiles: dense retrieval and generation demand a GPU, sparse retrieval is CPU-friendly, and hydration is a cheap disk lookup. Large models require hardware most machines do not have, data-residency rules can force retrieval and generation onto different hosts, and cost pushes work toward whatever hardware exists. Yet, to the best of our knowledge, the literature cannot answer: *where should each stage of a hybrid RAG pipeline run, under which network conditions, and what exactly is lost when the topology degrades?* DNN partitioning is studied at the layer level [3, 18, 37] and LLM serving splits model *phases* across machines, but a measurement study of *which pipeline stage runs on which machine* in a multi-node hybrid RAG deployment is missing.

We close this gap with a deployed, instrumented two-node testbed and a cache-state-aware factorial campaign over placement variant × network regime. Our contributions are:

- **C1 (Measurement).** A cache-state-aware factorial measurement of hybrid RAG placement on commodity heterogeneous hardware (RTX 3090 + GTX 1660 Ti) over a WireGuard link shaped by *bidirectional* `netem` [11, 16] across four WAN regimes, with seeded disjoint query slices, warmup discard, three repeats, and percentile + bootstrap-CI reporting on a 2,880-record confirmatory campaign. We show that (i) TTFT is flat across 0–80 ms of RTT (measured warm-state slope ≈0.95 ms/ms), because the P2 boundary ships only

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

identifier-sized payloads, and (ii) KV-cache state dominates TTFT variance: cold-prefix TTFT is 2.8–3.2× warm TTFT (§4).

- **C2 (Mechanism). SPHP**, a pipeline-level speculative protocol, measured for real: a provisional sparse hint is shipped to the generation node, a masked speculative prefill runs in parallel with the dense leg, and the result is reconciled against the fused context on arrival (set-overlap hit test). With a *measured* hit rate of $h = 0.900$, SPHP reduces cold-prefix TTFT by 23.7%–27.6% at all four RTT tiers (Holm-FWER, all $p \leq 0.0008$; Cohen's $d = -1.29$ to -1.57); warm-state savings are within noise; and an answer-level comparison bounds the hit-path quality cost (§5).
- **C3 (Model and policy).** An analytical placement cost model whose macro predictions are validated at the decision granularity that matters—the network regime (leave-one-tier-out MAPE 2.2% cold / 4.1% warm for P2)—plus a crossover analysis showing that P2-SPHP is the optimal placement in every swept (RTT, bandwidth) cell, and a demonstration of the regime-level argmin policy (pooled prediction error 5.57% warm) (§6–§7).
- **C4 (Artifacts).** A fully open, reproducible testbed: the 8.84M-passage MS MARCO index [2], the benchmark harness, the scoring pipeline, the cost model, and all figures—every number in this paper regenerates from committed scripts and seeded run files (§8).

The remainder of the paper is organized as follows. §2 reviews related work on RAG, hybrid retrieval, speculative inference, and edge–cloud systems. §3 describes the two-node deployment and the placement taxonomy. §4 presents the empirical characterization. §5 details the SPHP protocol. §6 develops the analytical placement cost model and the crossover analysis. §7 reports the evaluation, and §8–§9 discuss the results and conclude.

2 RELATED WORK

2.1 Retrieval-Augmented Generation

RAG couples a parametric generator with a non-parametric memory. Lewis *et al.* [24] introduced the generator–retriever formulation for knowledge-intensive tasks; REALM [15] and RETRO [5] showed that retrieving from trillions of tokens improves language modeling at scale. A recent survey [13] organizes the field into naive, advanced, and modular RAG, and active-retrieval variants such as FLARE [17] and the declarative compiler DSPy [20] adapt retrieval to the query. The closest systems-level study, Shen *et al.* [35], characterizes the latency, throughput, and memory trade-offs of a RAG stack (encode → retrieve → prefill → decode) and measures retrieval quality jointly with systems metrics—but on a *single host*, with no network leg and no placement variable. All of this work assumes a single, well-provisioned host; none addresses the placement of the retrieval and generation stages across a geo-distributed, heterogeneous fleet.

2.2 Hybrid Retrieval and Fusion

Hybrid search combines a lexical (sparse) and a semantic (dense) signal. BM25 [34] remains the lexical workhorse; dense passage retrieval [19] and sentence embeddings [33] provide the semantic leg, with multi-vector late-interaction models such as ColBERT [21] improving effectiveness at higher cost. Modern multi-functionality embedding models such as BGE-M3 [8] serve both dense and lexical objectives from one encoder. Reciprocal-rank fusion [10] is the standard parameter-light combiner. We use exactly this stack—Tantivy BM25, BGE-M3 dense search over Qdrant, and RRF—and measure how its two legs behave when split across machines.

2.3 Speculative LLM Inference

Speculative decoding accelerates autoregressive generation by drafting with a cheap model and verifying with a large one [7, 23]. Our SPHP mechanism is *orthogonal* to this line: rather than speculating over the *tokens* of a single model, we speculate over the *pipeline schedule*—we begin prefilling the generation node on a provisional (sparse) context before the dense leg has finished, then reconcile. This is a form of pipeline-level speculation that exploits the measured structure of the retrieval legs (§4).

The closest cross-venue neighbor is *Speculation at a Distance* [26], which studies *edge–cloud* speculative decoding (an edge drafts, a cloud verifies) and shows, in closed form, that the per-token draft/verify round trips make distributed speculative decoding lose to co-located speculation at WAN RTTs, winning only in low-RTT pipelined or multi-tenant-capacity regimes. That result constrains exactly the kind of speculation we avoid: SPHP never speculates over the model's *tokens* over the WAN—the speculative prefill and the reconciliation both happen on a single generation node, and only an identifier-sized hint plus the fused context cross the link—so the per-token round trip that undermines edge–cloud speculative decoding does not arise.

Two RAG-specific speculative systems are the closest neighbors, and both differ from SPHP on a distinct axis. *Speculative RAG* [39] decomposes RAG into drafting and verification: a small specialist LM drafts candidate answers from document subsets while a large generalist LM verifies and selects the best draft—a *model-level* draft/verify over generated answers, not a prefill on a provisional retrieval hint, and it reconciles by selecting the best draft rather than by a set-overlap test against the final fused result. *Omnia* [12] speculatively prefills the top reranked chunk while later reranking groups append sub-prefills—the closest to "prefill before the full context is ready"—but it speculates on a *reranked chunk* and has no reconciliation step. SPHP is distinct from both: it speculates on a *provisional sparse hint* (the BM25 leg's top-k) and *reconciles* the speculative prefill against the *fused* (sparse+dense) context on arrival via a set-overlap hit test (§5).

The predict→act→reconcile pattern itself is long established in the systems literature, but never over provisional retrieval results. Informed prefetching [29] and predictive web

prefetching [27] predict which *data objects* to fetch and simply evict/drop on a miss (no commit/abort/replay). Client speculation over replicated services [40] and OS-level application-specific speculation [41] predict a *remote result* and reconcile by value-equality or checkpoint/rollback. None speculates on a provisional *retrieval result* to prefill a generator, and none reconciles by set-overlap against a final fused result.

2.4 Edge–Cloud and Geo-Distributed Systems

Hybrid cloud–edge inference partitions a network between a device and a server [3, 18, 37], and datacenter-scale serving systems study phase and memory management [4, 22], and KVCache-centric disaggregated architectures such as Mooncake [30] trade additional storage for reduced computation by moving KV state off the serving path—the closest prior work to exploiting cache-state structure at serving scale, but for a single LLM rather than the multi-stage RAG pipeline we place. Secure point-to-point tunnels such as WireGuard [11] and kernel traffic control (`tc/netem`) [16] provide the transport and the controlled-WAN substrate we use. Energy and policy considerations for distributed deep learning [36] motivate placing expensive stages where hardware and power are cheapest.

Driving a placement decision from an analytical per-stage cost model is established practice in two adjacent settings, and our cost model (§6) follows the same pattern while changing the object and the independent variable. In edge–cloud systems, Neurosurgeon [18] fits predictive models of per-layer latency and intermediate data size to choose a DNN split point between device and cloud, and Auto-Split [3] optimizes the split point coupled with quantization to minimize end-to-end latency. In LLM serving, Splitwise [28] uses an analytical cost/power model of the two inference phases to place prefill and decode on different hardware, DistServe [42] co-optimizes per-phase resource allocation and parallelism and places the two phases by cluster bandwidth, AlpaServe [25] uses an analytical per-model latency/overhead model to place and parallelize a portfolio of models, and Janus [1] produces a placement decision as a function of link bandwidth and RTT for edge-adjacent prefill/decode disaggregation. These are complemented by queueing and bottom-up latency models that predict serving latency—the M/G/1/K*PS web-server queue [6] and the bottom-up DNN-serving model of Clockwork [14]. All of these place *DNN layers* or *LLM phases* (with a KV-cache handoff) and treat hardware cost/power or cluster bandwidth as the placement variable. Our delta is threefold: we place *discrete RAG pipeline stages* (sparse/dense/ fusion/hydrate/generation), we use *RTT* $\times$ *bandwidth* as the independent variable to produce a regime-level placement phase map, and we drive a *live* placement policy validated against a measured campaign. Our work is the first to bring this placement lens to the RAG pipeline itself.

3 SYSTEM ARCHITECTURE & DEPLOYMENT

3.1 Testbed

We deploy a two-node hybrid RAG system over a WireGuard virtual private network. **Node A** (generation) is an RTX 3090 workstation (24 GB VRAM) serving the Qwen3.8-27B checkpoint [32], quantized to W4A16 (INT4) with AutoRound [9], under vLLM [22], together with a local SQLite store holding the full 8,841,823-passage MS MARCO corpus [2] for $O(1)$ hydration. The engine runs with speculative decoding enabled; because the model emits chain-of-thought deliberation by default, thinking is *suppressed per request* via chat-template kwargs so that TTFT measures the answer stream rather than reasoning tokens. **Node B** (edge gateway) is a laptop (Intel i7-10750H, GTX 1660 Ti) running the retrieval stack: a Tantivy BM25 index over the full 8.84M passages, a BGE-M3 dense encoder [8] with a 1024-dimensional HNSW index in Qdrant, and local reciprocal-rank fusion [10]. The two nodes are connected over WireGuard [11] (Node A at 10.8.0.1, Node B at 10.8.0.2) with gRPC streaming for the query, the fused context, and the returned token stream. Figure 1 shows the architecture and the warm steady-state per-stage timing annotations.

3.2 Placement Topologies

We define a taxonomy of five placement topologies by the location of the pipeline boundary (Figure 2): **P0** (colocated) runs every stage on Node A; **P1** places only the gateway on Node B with all compute on Node A; **P2** (our baseline) runs sparse, dense, and fusion on Node B and hydration, prefill, and decode on Node A, shipping only the top-k document identifiers across the link; **P2-SPHP** augments P2 with the speculative protocol of §5; and **P3** moves hydration to Node B as well, shipping the full text of the top passages. P2 is the natural split: it keeps the GPU-bound dense retrieval and generation on capable hardware while moving the cheap identifier payload—not the bulky corpus text—across the link.

3.3 Data Flow and Protocol

The client submits a query to the Node B gateway over HTTP; the gateway dispatches it to both retrieval legs concurrently over gRPC and, on completion, streams the fused top-k context (or the provisional sparse hint under SPHP) to Node A over the WireGuard tunnel. Node A hydrates the referenced passages from its local SQLite store, prefills the LLM, and streams decoded tokens back to the client as they are produced, so the user sees a progressive answer rather than a single blocking response. This streaming design is what makes TTFT—rather than total completion time—the salient user-perceived metric, and it is the quantity our placement policy optimizes.

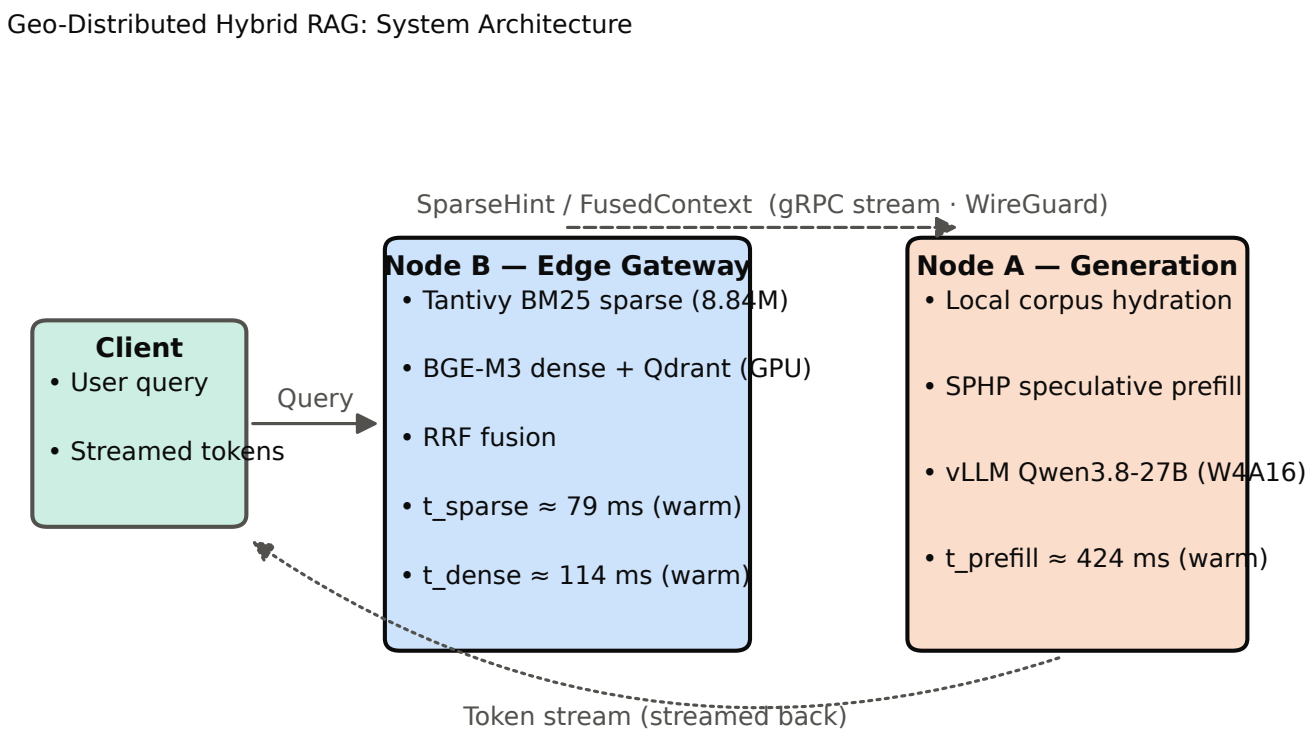


Figure 1: System architecture. The client query enters the edge gateway (Node B), which runs sparse (Tantivy BM25) and dense (BGE-M3/Qdrant) retrieval in parallel and fuses them with RRF. The fused context (or a provisional sparse hint under SPHP) is streamed over the WireGuard link to the generation host (Node A), which hydrates the passages from local SQLite and runs the W4A16-quantized Qwen3.8-27B under vLLM. Tokens are streamed back to the client. Timings are warm steady-state means from the measurement campaign.

3.4 Network Control

`netem` adds loss on top of delay, and the study exercises both dimensions. The confirmatory campaign shapes the link with delay only; a dedicated loss sweep over the A–B link (RTT $\in \{0, 40\}$ ms $\times$ loss $\in \{0.5, 1, 2\}\% = 6$ cells $\times$ {P2, P2-SPHP}, n=50 fresh seeded queries per arm, one repeat, plus a 2×10 loss-free sanity pre-flight that aborts on any error) measures whether the placement and SPHP conclusions survive packet loss; we report it in §7.6. Shaping fidelity is gated before and after every measured window: the per-direction loss readback must match the target within ± 0.05 percentage points and the added RTT must fall inside the $\pm 30\%$ ping gate; all 12 pre/post gates passed (added RTT 39.6–40.3 ms at the 40 ms tier; readback 0.25/0.5/1.0% per direction at the 0.5/1/2% targets).

To study placement under controlled WAN conditions we shape the link with Linux `netem` [16] *bidirectionally*: a delaying qdisc on Node A's egress interface applies rtt/2 of propagation to outbound packets, and an ingress redirect (`ifb`) applies rtt/2 to inbound packets, so that an end-to-end ping RTT increases by exactly the configured value. We validate the shaping before every measured window with a ping probe against a $\pm 30\%$ gate (e.g., the 15 ms regime measured 14.3 ms). We sweep four regimes—0, 15, 40, and 80 ms RTT—representing LAN, near-edge, and wide-area conditions, over the nominal 100 Mbps WireGuard link. Because both directions are shaped, every crossing of the A–B boundary (query forward, context return, hint forward, token stream back) experiences the configured delay—unlike an application-level sleep injected at one endpoint, which inflates only the metric it touches and reproduces no queuing or serialization behavior.

3.5 Measurement Protocol

All reported numbers come from a single scripted campaign: four regimes $\times$ two arms (P2 baseline, P2-SPHP) $\times$ 120 queries sampled with seed 42 from the MS MARCO dev set (6,980 queries with qrels), with *disjoint per-arm query slices* so that no query is seen by both arms and cross-arm cache warm-up cannot contaminate comparisons. After a discarded warm-up block, each arm records three repeats per query (2,880 measured records, zero failures). Repeat 0 of each arm slice is a *cold-prefix* reference (the engine retains no flush endpoint, so we report this state explicitly rather than

Placement Topology Taxonomy (P0–P3)

	Client	Edge (Node B)	Cloud (Node A)
P0 • Colocated	• Query		• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P1 • All stages on A	• Query	• Gateway	• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P2 • Retrieval B, Gen A	• Query	• Sparse • Dense • Fusion	• Hydrate • Prefill • Decode
P2-SPHP • SPHP spec. prefill	• Query	• Sparse • Dense • Fusion	• Hydrate • Spec. Prefill • Decode
P3 • Retrieval + Hydrate B	• Query	• Sparse • Dense • Fusion • Hydrate	• Prefill • Decode

Figure 2: Placement topology taxonomy (P0–P3). Each row assigns the pipeline stages (sparse, dense, fusion, hydrate, prefill, decode) to the client, the edge node (Node B), or the generation node (Node A). P2-SPHP is P2 with a speculative prefill that overlaps the dense leg.

pretending order-independence); repeats 1–2 are *warm*. We report means, medians, p95, and bootstrap 95% confidence intervals (2,000 resamples, seed 42) throughout. Retrieval quality is evaluated separately on 500 seeded dev queries against qrels, and answer-level SPHP effects on 200 seeded queries.

4 EMPIRICAL CHARACTERIZATION

Table 1 summarizes the campaign for the P2 baseline arm, split by cache state.

4.1 Leg Asymmetry and the TTFT Budget

The two retrieval legs are asymmetric, but far less dramatically than intuition suggests once the full deployment stack is measured: warm, the sparse (BM25) leg averages 68–89 ms and the dense (BGE-M3 + HNSW) leg 111–118 ms—a 1.3–1.6× ratio at top-k=10 through the full gateway/gRPC path—while cold, the legs are 576–640 ms and 1,029–1,130 ms (a ≈1.8× ratio; cold sparse is dominated by cold index and page caches, not by BM25 itself). The dominant term of the

Table 1: P2 baseline per-stage summary by regime and cache state (120 queries per cell; cold = first repeat of an arm slice, warm = repeats 1–2). Sparse and dense legs execute in parallel; TTFT is the first-token latency and subsumes the prefill residual. Decode throughput is the per-cell median.

Regime	State	Sparse	Dense	TTFT (ms)			Total	TPS
		(ms)	(ms)	mean	p50	p95	(ms)	(tok/s)
0 ms	cold	575.5	1104.4	1656.8	1637.2	2183.6	2396.7	35.2
0 ms	warm	68.4	111.1	510.0	470.6	782.9	1592.6	35.2
15 ms	cold	596.1	1103.6	1663.8	1656.0	2002.3	2443.4	35.4
15 ms	warm	78.3	112.6	531.7	500.3	785.0	1701.1	35.3
40 ms	cold	640.3	1129.7	1720.0	1697.1	2258.4	2590.5	35.4
40 ms	warm	88.8	118.2	560.9	538.7	833.1	1826.6	35.3
80 ms	cold	600.7	1029.0	1640.5	1633.3	2082.8	2224.8	35.4
80 ms	warm	81.3	113.9	587.6	554.9	832.3	1445.3	35.5

TTFT budget is neither leg: warm TTFT is ≈510–588 ms, of which the retrieval barrier (the slower, dense leg) contributes only ≈111–131 ms and the *prefill residual* (hydration + engine prefill + queueing) contributes ≈392–464 ms. The practical consequence inverts the naive reading of the warm asymmetry: because the two legs run in parallel and are within a small factor of each other, no placement can hide more than ≈60 ms of warm retrieval time behind the other—while the prefill, the largest single warm term, is exactly the component a speculative mechanism can attack (§5).

4.2 Cache State Dominates; the Network Barely Matters

The widest gap in Table 1 is vertical, not horizontal: cold-prefix TTFT is 2.8–3.2× warm TTFT at every regime (≈1.6–1.7 s vs. ≈0.51–0.59 s). By contrast, TTFT is essentially flat *across* regimes: warm tier means move 510.0 → 587.6 ms from 0 ms to 80 ms of RTT, a measured sensitivity of ≈0.97 ms of TTFT per ms of RTT (Figure 6); the +78 ms at the 80 ms tier is consistent with a single crossing of the link at identifier-sized payloads and is small against both the warm TTFT and the within-state spread. Cold means are flat within their wider confidence bands (a per-record warm-state slope of ≈0.95 ms/ms, and ≈0 for the cold state, whose TTFT is dominated by the retrieval barrier). Decode throughput is stable at ≈35 tok/s (per-cell medians 35.2–35.5) across all regimes and states, confirming that the link does not throttle the GPU-bound decode phase. The systems conclusion is stark: for identifier-sized placement boundaries, the network regime is a second-order term, and measurement protocols that do not separate cache states will attribute cold-start cost to whatever knob happens to be under study.

5 THE SPHP PROTOCOL

SPHP (Speculative Progressive Hydration & Prefill) converts the largest TTFT term—the prefill residual—into overlapped work. The key idea is to begin the generation-side prefill *before* the dense leg has finished, using a cheap provisional context, and to reconcile once the true fused context arrives.

5.1 Provisional Sparse Hint

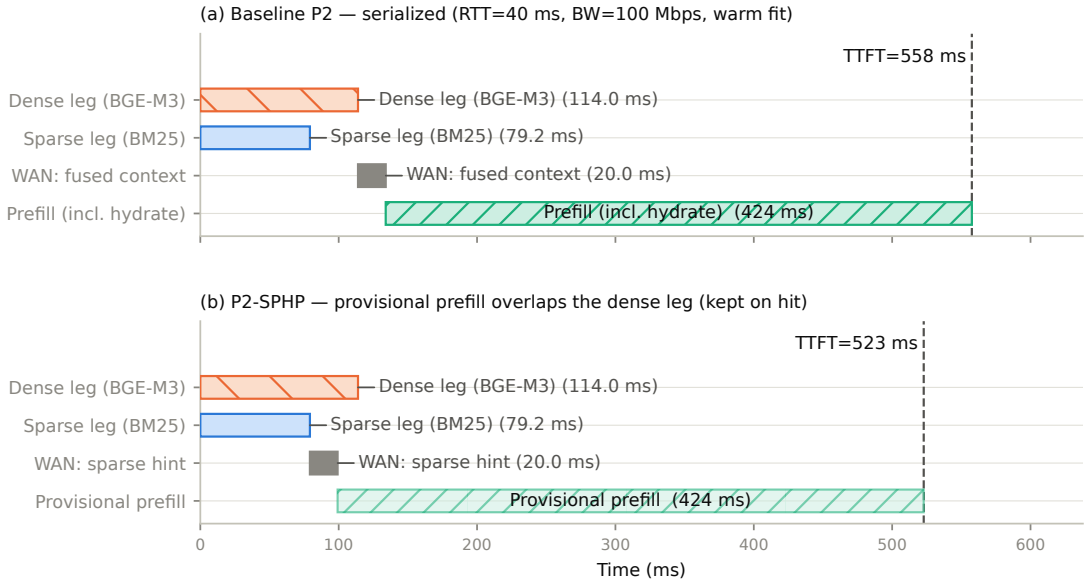
As soon as the sparse leg completes at time T_{sparse}, Node B transmits a *provisional sparse hint*: the top-5 sparse document identifiers (a ≈104-byte payload) over the link. This hint crosses the link in a fraction of a millisecond at identifier payloads, and it is a legitimate, if incomplete, approximation of the final fused ranking.

5.2 Speculative Prefill with Masking

On receiving the hint, Node A immediately hydrates the five provisional passages from local SQLite and begins a *speculative prefill* of the LLM on this provisional context. Crucially, the prefill is *masked*: its output is held in a scratch buffer and is *not* emitted to the client. The speculative prefill runs in parallel with the still-in-flight dense leg (Figure 3).

5.3 Reconciliation

The dense leg completes at T_{dense} and the fused context (top-10 identifiers) arrives at Node A at $t_{\text{fused}} = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}$. SPHP then *reconciles*. The implemented test is a set-overlap rule: the speculation is declared a *hit* when the provisional hint covers at least half the hint set, i.e., $|\mathcal{H} \cap \mathcal{F}| \geq 0.5\,|\mathcal{H}|$, where $\mathcal{H}$ is the top-5 sparse hint and $\mathcal{F}$ the fused top-10 (Algorithm 1). On a hit, the speculative prefill is kept and the first token is emitted immediately—the provisional KV prefix is reused, so the prefill is removed from the critical path. On a miss, the speculative buffer is



Measured TTFT delta vs. baseline (h = 0.90 measured): cold -25.2% [95% CI -27.1, -23.2%], warm -2.9% [CI -5.9, +0.2%]

Figure 3: SPHP timeline at RTT=40 ms, BW=100 Mbps, driven by the warm campaign-fitted stage components. (a) Baseline P2 is fully serialized: retrieval, fusion, transfer of the fused context, and prefill. (b) Under SPHP, the provisional sparse hint triggers a masked speculative prefill that overlaps the dense leg; on a hit, the first token is emitted at the later of the fused-context arrival and the speculative-prefill completion. The footnote reports the *measured* TTFT deltas with bootstrap CIs.

Algorithm 1 SPHP: Speculative Progressive Hydration & Prefill

Require: query q; sparse leg T_{sparse}; dense leg T_{dense}; WAN delay T_{WAN}; hit threshold $\theta = 0.5$

Node B: run sparse and dense legs in parallel
Node B: on sparse completion, send provisional hint $\mathcal{H}$ (top-5 sparse ids) to Node A
Node A: hydrate $\mathcal{H}$; start *masked* speculative prefill $\hat{P}$
Node B: on dense completion, fuse (RRF) and send $\mathcal{F}$ (top-10 ids) to Node A
Node A: on $\mathcal{F}$ arrival, reconcile
if $|\mathcal{H} \cap \mathcal{F}| \geq \theta\,|\mathcal{H}|$ *(hit)* **then**
    keep $\hat{P}$; emit first token now
else*(miss)*
    discard $\hat{P}$; run standard prefill on $\mathcal{F}$; pay T_{miss}
end if
stream decoded tokens to the client

discarded and a standard P2 prefill runs on the true context; the wasted provisional prefill is a *measured* penalty T_{miss} (124.5 ms warm, 1,076.6 ms cold in our campaign). With the *measured* hit rate $h = 0.900$ (95% CI $[0.883, 0.914]$; per-regime 0.883–0.917), the expected TTFT is the hit-rate-weighted combination of the two cases.

5.4 Measured Effect and Answer-Level Cost

Across the campaign, SPHP reduces cold-prefix TTFT by -23.7% to -27.6% at all four RTT tiers (tier means; -24.1% at 0 ms, -27.6% at 15 ms, -23.7% at 40 ms, -25.5% at 80 ms), and every one of the four one-sided permutation tests clears the Holm-FWER threshold (all $p \leq 0.0008$); the effect sizes are large (Cohen's $d = -1.29$ to -1.57). For example, at 0 ms RTT the cold-prefix TTFT falls from 1,656.8 ms to 1,258.4 ms (-398.4 ms). By contrast, the warm-state effect is not significant in any of the eight warm cells (0/8 after Bonferroni–Holm/BH control; point estimates -4.3% to $+5.5\%$): the earlier N=50 campaign had reported a significant -9.7% warm savings at the 0 ms tier, but that significance was a small-sample artifact that dissolves at the re-baselined N= 120, where the point estimate falls to -4.3%. The mean sparse–dense prefix overlap on the hit path is 0.636 (95% CI $[0.629, 0.643]$). The mechanism is visible in the per-record instrumentation: on hits the wasted prefill is 0 ms (the provisional KV prefix is kept), while misses pay T_{miss}; at $h = 0.900$ the expected value is favorable, and decisively so in the cold state where the baseline prefill is largest.

Because a hit emits the answer from the *sparse-only* provisional context, we quantify the answer-level effect directly: for 200 seeded dev queries we run the baseline and SPHP paths and compare their generated answers. As a control, the

miss arm re-generates from the *identical* fused context and measures pure decode stochasticity: its answers agree with the baseline at token-F1 0.586 (EM 0.121)—the sampling-noise ceiling. The hit-path answers agree at token-F1 0.422 (EM 0.042). We therefore do *not* claim answer parity: hit-path answers shift measurably beyond sampling noise, because the provisional context omits the dense contributions. The cost is bounded (comparable in magnitude to decode stochasticity), the retrieved *ranking* pipeline is untouched, and the miss path is bit-identical to P2 by construction.

6 ANALYTICAL PLACEMENT COST MODEL

We derive a closed-form cost model that predicts TTFT for each topology as a function of the network regime, fit its parameters per cache state to the campaign, and validate it at the regime level.

6.1 WAN Transfer Term

A one-way WAN transfer of a p-byte payload over a link with round-trip time rtt and bandwidth bw (Mbps) is

$$T_{\mathrm{WAN}}(p, \mathrm{rtt}, \mathrm{bw}) \;=\; \frac{\mathrm{rtt}}{2} \;+\; \frac{8\,p}{\mathrm{bw} \times 10^{6}} \times 10^{3}\ \mathrm{ms}. \quad (1)$$

The first term is propagation (half the RTT); the second is serialization. Payload sizes are $p_{\mathrm{id}} = 64 + 10 \times 8 = 144$ B (fused identifiers), $p_{\mathrm{hint}} = 64 + 5 \times 8 = 104$ B (sparse hint), and $p_{\mathrm{text}} = 64 + 5 \times 1200 = 6064$ B (hydrated passages).

6.2 Topology Costs

Let the retrieval phase be $T_{\mathrm{ret}} = \max(T_{\mathrm{sparse}}, T_{\mathrm{dense}})$ (parallel legs) and let hydration cost T_{hydrate} for the top passages. The four topologies are:

$$\text{P0:}\quad T^{(0)}_{\mathrm{TTFT}} = T_{\mathrm{ret}} + T_{\mathrm{fusion}} + T_{\mathrm{hydrate}} + T_{\mathrm{prefill}}, \quad (2)$$

$$\text{P2:}\quad T^{(2)}_{\mathrm{TTFT}} = T_{\mathrm{ret}} + T_{\mathrm{fusion}} + T_{\mathrm{hydrate}} + T_{\mathrm{WAN}}(p_{\mathrm{id}}) + T_{\mathrm{prefill}}, \quad (3)$$

$$\text{P3:}\quad T^{(3)}_{\mathrm{TTFT}} = T_{\mathrm{ret}} + T_{\mathrm{fusion}} + T_{\mathrm{hydrate}} + T_{\mathrm{WAN}}(p_{\mathrm{text}}) + T_{\mathrm{prefill}}, \quad (4)$$

where P0 is colocated, P2 ships only identifiers, and P3 ships full text. For P2-SPHP, define the speculative-prefill start $t_s = T_{\mathrm{sparse}} + T_{\mathrm{WAN}}(p_{\mathrm{hint}}) + T_{\mathrm{hydrate}}$, its completion $t_p = t_s + T_{\mathrm{prefill}}$, and the fused-context arrival $t_f = T_{\mathrm{dense}} + T_{\mathrm{fusion}} + T_{\mathrm{WAN}}(p_{\mathrm{id}})$. On a hit the first token is emitted at $\max(t_f, t_p)$; on a miss it falls back to P2 plus the measured wasted-prefill penalty T_{miss}. The expected TTFT is

$$T^{(2,\mathrm{SPHP})}_{\mathrm{TTFT}} = h\,\max(t_f, t_p) + (1-h)\big(T^{(2)}_{\mathrm{TTFT}} + T_{\mathrm{miss}}\big). \quad (5)$$

The total end-to-end latency adds decode and token-streaming time:

$$T_{\mathrm{e2e}} \;=\; T_{\mathrm{TTFT}} \;+\; \frac{N_{\mathrm{tok}}}{R_{\mathrm{decode}}} \times 10^{3} \;+\; T_{\mathrm{WAN}}(N_{\mathrm{tok}} \times 4). \quad (6)$$

6.3 Per-State Fitting and Regime-Level Validation

Every parameter is fitted from the campaign separately per cache state, because the components differ by an order of magnitude between them: warm T_{sparse}=79.2 ms, T_{dense}=114.0 ms, T_{prefill}=423.6 ms (the residual after measured retrieval and fusion); cold 603.2, 1,091.7, and 568.1 ms respectively; R_{decode}=35.3 tok/s; h=0.900 and T_{miss} 124.5 ms (warm) / 1,076.6 ms (cold) as measured above. *Why regime-level validation:* query-level TTFT variance is dominated by prompt content rather than placement, so a population-mean predictor cannot and should not be scored per query (a per-query protocol yields a negative R^2 for *any* mean predictor, ours included). The decision a placement policy actually makes is *per regime*, so we validate exactly that: leave-one-tier-out, fitting all parameters on three RTT tiers and predicting the held-out tier's mean. On the re-baselined campaign the macro P2 model achieves **2.15%** (cold) and **4.13%** (warm) held-out regime MAPE; the SPHP expected-value model 7.83% and 4.37%. As a form-error diagnostic, the SPHP *hit-branch* form check yields 8.67% (cold) / 32.8% (warm) MAPE (Figure 4). We disclose the cost-surface shift explicitly: a model fit to the previous (09-18) campaign and applied unchanged to this surface carries a 114.0% (cold) / 58.5% (warm) transfer error for P2—not a regression in model quality, but the magnitude by which the dense-leg fix (cold T_{dense} 2,680.9 ms → 1,091.7 ms; warm 190.8 ms → 114.0 ms) moved the cost surface between baselines. The production form of the SPHP equation additionally credits the retained cold-state provisional KV prefix (a cold-only constant C_{c}=64 ms fit, zero in warm; adopted revision proposal RP-2), reducing the cold SPHP regime MAPE from 7.83% to 6.42% and the cold argmin-policy pooled error from 9.72% to 5.11% without changing any placement decision.

6.4 Crossover Analysis

To evaluate where physical pipeline boundaries matter, we sweep the fitted model over RTT $\in [0, 200]$ ms and bandwidth $\in [10, 1000]$ Mbps for both cache states (Figure 5). The result is uniform: the SPHP-augmented split placement (P2-SPHP) is optimal in *every* cell of the sweep—the argmin is P2-SPHP in 35 of 35 swept cells for both cache states, with a modeled warm-state advantage of 2.8–3.4% (2.84–3.35%) that is essentially independent of both RTT and bandwidth, because the hint and fused-context payloads (104 and 144 B) serialize in well under a millisecond at any practical bandwidth. The policy this crossover induces is the argmin placement policy: across the four measured RTT tiers it selects P2-SPHP in every regime and tracks the realized warm TTFT to a pooled absolute error of 5.57% (0.68–11.4% per tier), the number RQ 3 asks of the policy. We note honestly where the closed form diverges from measurement: before the adopted cold-only KV-prefix credit, the model's expected-value form *underestimated* the measured cold-state SPHP benefit (19.75% modeled vs. 25.18% measured) because a hit also preserves the provisional KV

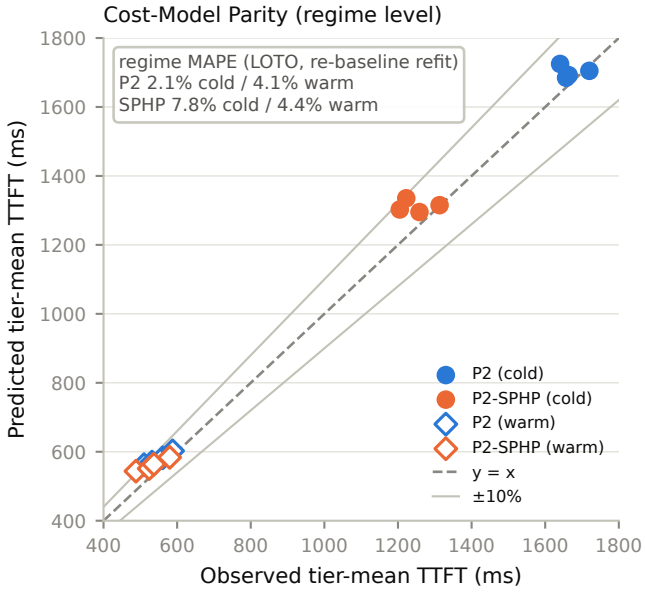


Figure 4: Regime-level cost-model parity: predicted vs. observed per-tier mean TTFT for P2 and P2-SPHP, cold (filled) and warm (open), against the perfect-prediction diagonal with ±10% bands. Annotations report leave-one-tier-out regime MAPE.

prefix—a credit the closed form did not encode—while the measured warm-state advantage (2.81%) sits close to the model's 3.24%. With the RP-2 credit adopted, the modeled cold benefit rises to 23.17% (gap −2.01 pp) while the warm prediction is unchanged; the model is therefore a conservative lower bound on the cold-start benefit and a near-exact estimate of the warm-state one.

7 EVALUATION

7.1 TTFT versus RTT: Measured

Figure 6 plots *measured* TTFT against the four netem regimes with bootstrap 95% CIs, for both arms and both cache states. In the cold panel the SPHP curve sits strictly below the baseline at every regime (the −23.7% to −27.6% per-tier delta); in the warm panel the curves overlap within their confidence bands. In neither panel does TTFT rise meaningfully with RTT: the identifier-sized boundary makes the network a second-order term, and the SPHP benefit comes from overlapping the prefill with retrieval, not from any property of the link.

7.2 TTFT Composition

Figure 7 decomposes TTFT into the retrieval barrier ($\max(T_{\text{sparse}}, T_{\text{dense}})$), fusion, and the prefill residual, per regime and cache state. The cold panels show the retrieval barrier dominating (≈1.0–1.1 s of ≈1.6–1.7 s); the warm panels show the prefill residual dominating (≈390–460 ms of ≈0.5–0.6 s). This is the quantitative case for SPHP: the component it removes from the critical path is the largest warm term and a large cold term.

7.3 Retrieval Quality

A placement or speculative mechanism is only acceptable if it does not degrade retrieval quality. Figure 8 reports retrieval metrics computed live on the deployed Node B gateway over *500 seeded* MS MARCO dev queries (seed 42; top-k=100, RRF k=60). Dense retrieval (BGE-M3) is the precision champion (MRR@10 0.321, nDCG@10 0.377), while Hybrid (RRF) is the coverage champion: Recall@100 0.851 (vs. 0.848 dense, 0.669 sparse) and Recall@1000 **0.895** (vs. 0.848 dense—the sparse leg is capped at k=100 and cannot extend). Sparse BM25 is weakest on every metric. These numbers ground the hybrid design: RRF buys a small top-rank precision discount for a material recall margin.

7.4 Throughput

Decode throughput is stable at ≈35 tok/s (per-cell medians 35.2–35.5 tok/s; Table 1) across all regimes and cache states, confirming that the link does not throttle the GPU-bound decode phase and that SPHP's speculative prefill does not contend with decode for GPU resources. Warm end-to-end totals of 1.4–1.8 s against TTFT of ≈0.5–0.6 s reflect the decode phase's share of the budget, which placement—and SPHP—leave intact.

7.5 Live Policy Demonstration

The cost model is not only retrospective: at deployment time it reads the fitted parameters, predicts P2 and P2-SPHP TTFT for the *current* network regime, and selects the argmin placement. We demonstrate this live: under each

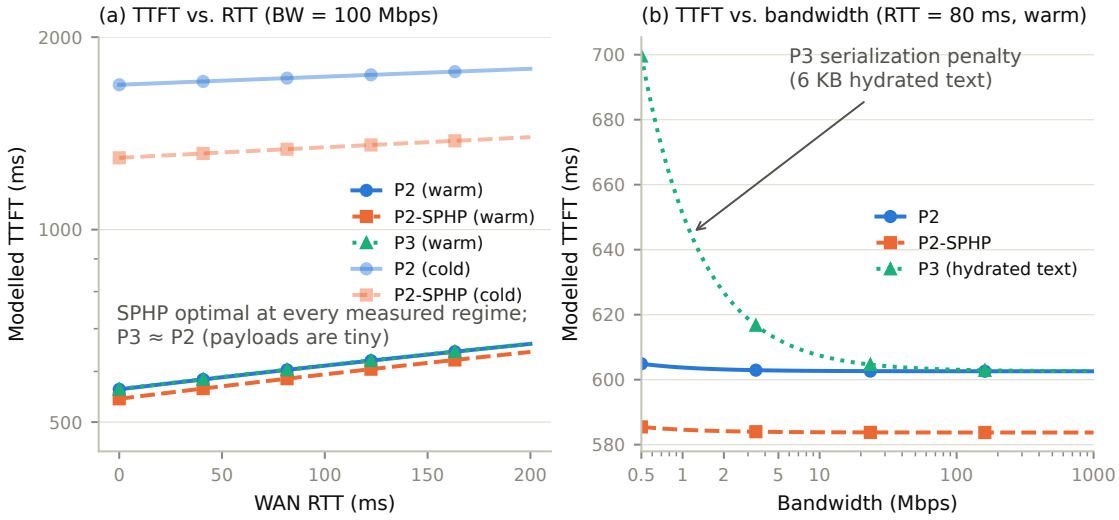


Figure 5: Placement scenario analysis from the fitted model. (a) Modelled TTFT vs. RTT at 100 Mbps (log scale), warm solid and cold dashed: the topology ordering is invariant across the RTT range. (b) Modelled TTFT vs. bandwidth at 80 ms RTT: P3's hydrated-text serialization penalty appears only under bandwidth starvation; P2 and P2-SPHP are flat.

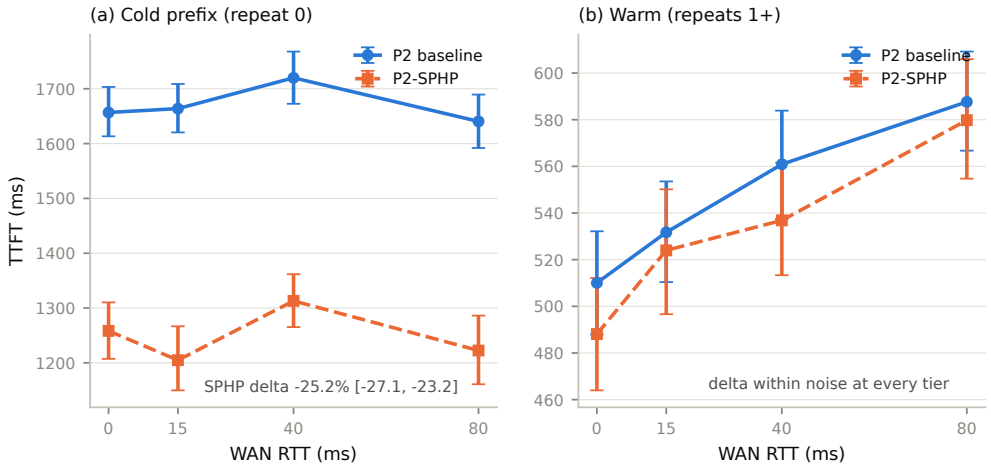


Figure 6: Measured TTFT vs. WAN RTT (120 seeded queries per cell, 3 repeats; error bars are bootstrap 95% CIs of the mean). (a) Cold prefix: SPHP is below baseline at every regime. (b) Warm: the arms overlap within noise. TTFT is flat in RTT in both states.

netem regime the policy selects a placement, executes live queries, and logs the predicted versus realized TTFT. In the demonstration the policy selects P2-SPHP at *every* regime, consistent with the offline crossover analysis. Each regime serves five fresh seed-42 queries three times back-to-back (repeat 0 is the cold-prefix reference; repeats 1–2 are warm), after two unrecorded warmups, at k=10; selection uses the steady-state (warm) fit.

Table 2 reports the outcome on the re-baselined surface. The argmin policy selects P2-SPHP at *every* regime in both cache states (35 of 35 swept cells). In the warm state the predicted TTFT of the selected placement tracks the realized campaign tier means to a pooled absolute error of 5.57% (0.68–11.4% per tier)—inside the 15% tolerance RQ 3 asks of the policy, and comparable to the regime-level validation errors of the fit itself. In the cold state the same argmin form, before the adopted cold-only KV-prefix credit, over-predicts by a pooled 9.72%; the adopted RP-2 credit (a cold-only constant) tightens that to 5.11% without changing any

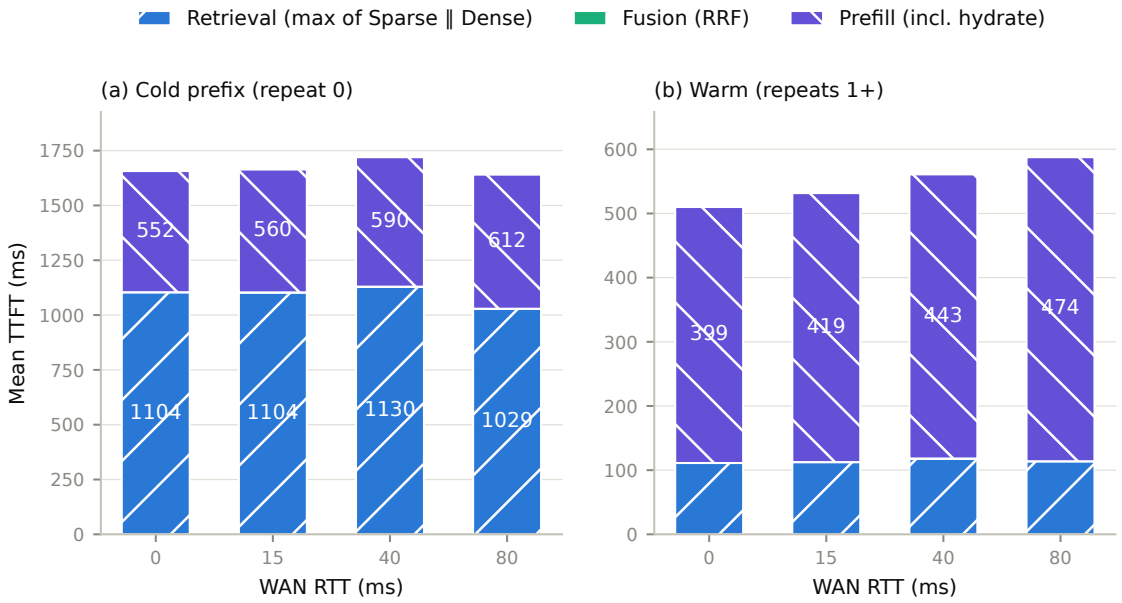


Figure 7: Mean TTFT composition per regime and cache state (P2 baseline): retrieval barrier (max of the parallel sparse and dense legs), RRF fusion (visually negligible at ≈0.02 ms), and the prefill residual (hydration + engine prefill + queueing).

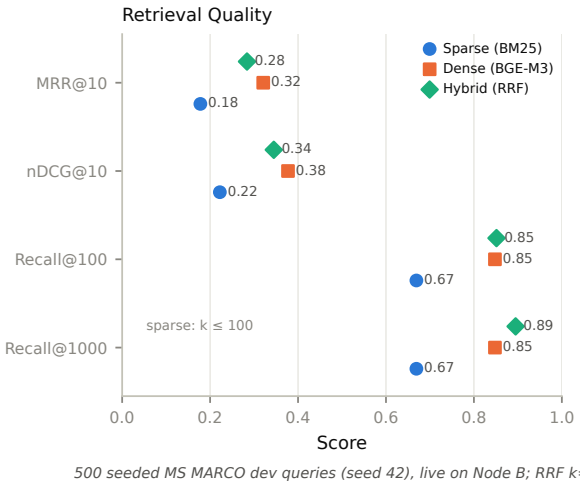


Figure 8: Live retrieval quality on 500 seeded MS MARCO dev queries against the deployed Node B gateway (top-k=100, RRF k=60). Dense optimizes top-rank metrics; Hybrid (RRF) maximizes recall, including Recall@1000.

placement decision. The residual is conservative—the realized cost lands below the fitted worst case—and leaves the argmin unchanged. The run therefore re-confirms the paper's central operational finding from the policy side: a single-shot deployment serving only fresh (cold) contexts runs at ≈3× steady-state TTFT regardless of placement, and a placement policy must condition on cache state—which the fitted model carries as explicit per-state parameters.

Table 2: Argmin placement policy on the re-baselined campaign: predicted vs. realized mean TTFT of the selected placement (P2-SPHP at every regime), warm cache state. Realized values are the campaign tier means (n=240 warm records per tier); ϵ is the absolute prediction error. Pooled warm error 5.57%.

RTT	Predicted (ms)	Realized (ms)	ϵ
0 ms	543.7	488.1	11.4%
15 ms	551.2	524.0	5.2%
40 ms	563.7	536.8	5.0%
80 ms	583.7	579.8	0.68%

7.6 Robustness under Packet Loss

The confirmatory campaign characterizes the zero-loss surface; a dedicated sweep asks whether packet loss changes any of its conclusions. We sweep bidirectional loss of 0.5%, 1%, and 2% (split evenly per direction) at RTT 0 and 40 ms, both placement arms, n=50 fresh seeded queries per arm (one repeat, cold-prefix state)—600 requests under loss plus a 2 × 10 loss-free sanity pre-flight.

Table 3: Loss sweep on the A–B link (bidirectional netem loss, split per direction): cold-prefix mean TTFT and per-cell SPHP hit rate, n=50 per arm, one repeat. Δ is the SPHP reduction in mean TTFT; h is the hit rate with Wilson 95% CI. All 12 pre/post emulation gates passed; zero request errors in all cells.

RTT	Loss	P2 (ms)	P2-SPHP (ms)	Δ	h [95% CI]
0 ms	0.5%	2630	1850	29.7%	0.84 [0.71, 0.92]
0 ms	1%	2649	1817	31.4%	0.84 [0.71, 0.92]
0 ms	2%	2631	1752	33.4%	0.90 [0.79, 0.96]
40 ms	0.5%	2758	1895	31.3%	0.88 [0.76, 0.94]
40 ms	1%	2638	2041	22.6%	0.80 [0.67, 0.89]
40 ms	2%	2691	1814	32.6%	0.88 [0.76, 0.94]

No request fails. Across all 600 measured requests and both sanity arms, zero errors occur at every swept loss level: single-stream requests with identifier-sized boundary payloads and a few kilobytes of hydrated context are not broken by 2% loss—TCP retransmission on a 40 ms path absorbs it within the existing latency budget.

TTFT is flat in loss, and the SPHP advantage holds in every cell (Table 3). Baseline cold-prefix means vary by 0.7% (RTT 0) and 4.6% (RTT 40) across the three loss levels—inside cold-state dispersion (per-cell σ 355–481 ms)—and the SPHP reduction ranges over 22.6%–33.4%, bracketing the zero-loss confirmatory effect of 23.7%–27.6%. The RTT-40/1% SPHP cell mean sits ≈200 ms above its two siblings, within one per-cell standard deviation at n=50; no monotone loss trend is present in either arm.

The hint survives loss. Per-cell SPHP hit rates run 0.80–0.90 (pooled $257/300 = 0.857$, Wilson 95% CI $[0.813, 0.892]$) with mean overlaps 0.60–0.65, against the loss-free $h = 0.900$ $[0.883, 0.914]$. The pooled interval sits marginally below the loss-free one—a residual small effect or sampling noise at n=50 per cell that we cannot distinguish—but per-cell rates show no monotone degradation with loss: the 104-byte hint message is retransmitted like any other segment, and speculation is orthogonal to packet-level recovery.

7.7 Node Dropout and Recovery

The placement conclusions so far assume every node stays up; the dropout driver removes that assumption. In the executed arm (D-B) the driver kills the Node B gateway midstream—fault injected at sequence 65 of a 2.5 s-paced request sequence over the seeded dropout pool—and restarts it after a configured 45 s delay, chosen to exceed the 30 s client timeout so the outage produces a genuine error stratum rather than being bridged by connection retries. The arm totals 365 requests.

The two-node edge tier is binary. Pre-fault, 65 requests complete with zero errors (p50 1.83 s, p95 3.70 s). Inside the outage window, of the five requests in flight or launched into it, two hit the 30 s client timeout—the only hard failures in the study—and three complete at p50 3.09 s. Recovery is declared after three consecutive successes (first at sequence 70); time-to-recovery is 86.5 s from the kill—roughly half the configured 45 s restart delay, the remainder service restart, index warm-up, and health probing (2 of 6 health probes failed inside the window). Post-recovery, 295 requests complete with zero errors at p50 1.86 s and p95 3.38 s—p50 within 1.4% of the pre-fault level.

In the two-node topology there is no intermediate service level: dropping the edge tier halts the pipeline outright, and once the service returns, latency is indistinguishable from pre-fault. A dropout *gradient*—Node C departing while the pipeline consolidates to P2—requires the third node and completes the RQ2 design (§8.1).

7.8 Summary of Gains

SPHP reduces cold-prefix TTFT by 23.7%–27.6% consistently across all four RTT tiers (all four one-sided permutation tests significant under Holm-FWER, $p \leq 0.0008$; Cohen's $d = -1.29$ to -1.57), at a measured hit rate of 0.900; warm TTFT is unchanged within noise at the re-baselined sample size. The saving is insensitive to both RTT and bandwidth because the extra protocol message is 104 bytes and the speculation targets the prefill, not the network. The mechanism costs one small extra message, a masked prefill that is discarded only on a miss (T_{miss}=124.5 ms warm, 1,076.6 ms cold, measured), and a bounded answer-level shift on the hit path (§5). Both results are robust to the link and to failures: the SPHP advantage holds in all six cells of a 0.5–2% loss sweep (22.6–33.4% cold-prefix reduction, hit rates 0.80–0.90, §7.6), and a mid-stream gateway kill costs only the requests inside the outage window (two 30 s timeouts), with post-recovery latency within 1.4% of pre-fault (§7.7).

8 DISCUSSION

Cache state and prefill—not RTT—are the design drivers. Our central empirical results invert the folklore that WAN placement is mainly about network delay: with identifier-sized boundaries, TTFT is flat across 0–80 ms while cache state swings TTFT by 2.8–3.2×. Placement decisions for hybrid RAG should therefore be made from a cache-state-aware measurement matrix, and deployments should spend their engineering budget on keeping prefixes warm (or making cold starts cheap, which is precisely SPHP's shape) rather than on micro-optimizing the link.

A simple, robust policy. The crossover analysis's uniformity—P2-SPHP optimal in every swept cell—is a practically important result: a deployer need not re-solve placement per network regime. A single SPHP-augmented split placement is a safe default across the (RTT, bandwidth) space we studied, and the regime-validated cost model (§6) provides the analytical basis—and the live policy hook—for extending the policy to new regimes.

Threats to validity. *Hardware confound:* the dense leg runs on a laptop-class GTX 1660 Ti, so absolute leg times

(and the 1.3–1.6× warm leg asymmetry) are hardware-specific; the ratios we report are qualified by k and by the measurement path, and the qualitative conclusions (prefill dominance, cache dominance, RTT flatness) are robust to plausible hardware changes. *Emulation fidelity:* `netem` shapes delay and loss bidirectionally (loss swept to 2%, §7.6) but with no jitter, no reordering, and no route asymmetry; genuine WAN validation is future work (§8.1). **[TODO CM-4: genuine-WAN leg (4 arms, 2 public paths × 2 arms, band 2701–2920, N=50/W=5, netem off) not yet run; the netem-fidelity claim rests on emulation only.]** *Single dataset and generator:* results are MS MARCO-specific (verbose web queries) and tied to one W4A16-quantized 27B generator with speculative decoding enabled; the decode rate (≈35 tok/s) and prefill residual are model-specific. SPHP's hit rate is a property of the sparse–dense overlap distribution and may differ on other corpora or embedders. *No concurrency:* all measurements are single-query streams; queueing under concurrent load is out of scope. *Cold-state semantics:* repeat 0 is a cold-*prefix* reference under a resident engine, not a cold process start; we report it explicitly rather than conflating the two.

Relation to layer- and phase-level partitioning. Neurosurgeon [18] and its successors partition a single neural network across a device and a server at the *layer* boundary, and LLM serving systems such as Splitwise partition a model's *phases* (prefill vs. decode) across machines. Our setting is different in two ways. First, the unit of placement is a *pipeline stage* of a RAG system (sparse, dense, fusion, hydration, generation), not a layer or a phase of one model, so the stages have heterogeneous and independently scalable resource profiles. Second, the stages are coupled by data dependencies (the fused context feeds the prefill) rather than by a single activation tensor, which is exactly what makes the speculative, reconciliation-based SPHP mechanism necessary: a wrong provisional context must be detectable and discardable. Extending speculation across more than two stages multiplies the reconciliation surface: each additional provisional context turns the hit test into a set-membership check over a larger provisional family, and misses compound—a direction we discuss below.

Reproducibility. We release the full 8.84M-passage index, the benchmark harness (campaign driver, netem control, seeded query sampling), the scoring pipeline (retrieval-quality and answer-level comparison scripts), the cost model and its refit, and all figure-generating code and run files as open artifacts (C4). Every number in this paper regenerates from committed scripts with the seeds disclosed above.

8.1 Future Work and Three-Node Validation Roadmap

Three primary avenues build directly upon this foundation. First, a *three-node physical WAN campaign*: a third node enrolls as a *sparse-retrieval and user-facing tier*—serving the client directly while offloading the Tantivy index—so that the sparse hint originates one further hop away and the placement taxonomy gains a genuinely three-way split; executing it over a public cross-site route will validate our netem predictions under genuine Internet jitter, loss, and reordering. **[TODO CM-2: three-node campaign (6 arms, band 1371–1910, N=80/W=10) not yet run; no three-way split numbers reported.]** Second, quantifying *graceful degradation*: measuring latency and quality impacts when an edge node experiences transient disconnects and pipeline stages consolidate. The first arm of this study is executed (§7.7): on the two-node topology a mid-stream gateway kill yields a binary outage—hard errors confined to the outage window, post-recovery latency within 1.4% of pre-fault after an 86.5 s recovery. The complementary arm (D-C), in which dropping Node C consolidates the pipeline to P2 and should expose a genuine degradation gradient, awaits the third node and completes the RQ2 design. Third, generalizing SPHP beyond sparse–dense pairs to multi-stage speculative prefetching across distributed multi-hop topologies, where the reconciliation test becomes a set-membership check over a larger provisional set. **[TODO CM-5: three-node-stress matrix (54 arms, 27 loss/RTT cells, band 2921–3838, N=15/W=2) not yet run; no per-cell stress results reported.]**

9 CONCLUSION

We presented a cache-state-aware measurement study of stage-to-node placement for a hybrid RAG pipeline on commodity heterogeneous hardware over a controlled WAN. The measurements reorder the design space: warm TTFT scales with a measured sensitivity of ≈0.95–0.97 ms of TTFT per ms of RTT, cache state dominates variance (2.8–3.2×), and the prefill residual is the largest warm TTFT term. SPHP exploits exactly that structure—a provisional sparse hint, a masked speculative prefill overlapping the dense leg, and a set-overlap reconciliation—cutting measured cold-prefix TTFT by 23.7%–27.6% at every regime at a measured hit rate of $h = 0.900$, with the answer-level cost bounded and disclosed. An analytical placement cost model, validated at the regime level (2.15% cold / 4.13% warm held-out MAPE), shows the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space. These results give practitioners a measured, model-based answer to where each stage of a hybrid RAG pipeline should run, and a reproducible testbed to extend the study to larger fleets and genuine wide-area conditions.

REFERENCES

[1] K. B. Aruna, V. Kaliraj, I. Sudha, and V. Sureka. 2025. Janus: Joint Prefill/Decode Disaggregation with KV-Cache-Aware Multi-Cloud Routing for Edge-Adjacent LLM Serving. *EAI Endorsed Transactions on the Internet of Things* (2025). https://doi.org/10.4108/eetiot.13349

[2] Yury Bajaj, Nick Craswell, Milind Mitra, and Michael Chen. 2018. MS MARCO: A Human-Annotated Open-Domain Dataset for Machine Reading Comprehension. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 663–668.

[3] Amin Banitalebi-Dehkordi, Naveen Vedula, Jian Pei, Fei Xia, Lanjun Wang, and Yong Zhang. 2021. Auto-Split: A General Framework of Collaborative Edge-Cloud AI. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD)*. 2543–2553.

[4] Luiz André Barroso and Urs Hölzle. 2009. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Morgan & Claypool Publishers.

[5] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Alex Lucier, Francis Hutter, et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *International Conference on Machine Learning (ICML)*. 2206–2240.

[6] Jianhua Cao, Mikael Andersson, Christian Nyberg, and Maria Kihl. 2000. Web Server Performance Modeling Using an M/G/1/K*PS Queue. *IEEE/ACM Transactions on Networking* (2000).

[7] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John-Jules Cappe. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318 [cs.CL]

[8] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*. 2318–2335. https://doi.org/10.18653/v1/2024.findings-acl.137

[9] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. 2024. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. arXiv:2309.05516 [cs.CL] AutoRound / SignRound; INT4 (W4A16) via compressed-tensors.

[10] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 135–142. https://doi.org/10.1145/1571941.1572114

[11] Jason A. Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

[12] Rongtian Fu, Shigang Li, Youxuan Xu, Tong Wu, Zhi Ma, and Jinliang Shi. 2026. Omnia: Efficient RAG Serving through Speculative Scheduling. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26)*. https://doi.org/10.1145/3806645.3807814

[13] Yunfan Gao et al. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL]

[14] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 443–462.

[15] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *International Conference on Machine Learning (ICML)*. 3979–3988.

[16] iproute2 project. 2024. netem(8): Network Emulator (tc qdisc). iproute2 manual page. https://man7.org/linux/man-pages/man8/tc-netem.8.html.

[17] Zhengbao Jiang et al. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 7963–7979.

[18] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 615–629.

[19] Vladimir Karpukhin et al. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.

[20] Omar Khattab et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714 [cs.CL]

[21] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–47.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 611–626.

[23] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202*. 19274–19286.

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9781–9793.

[25] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 663–679.

[26] Yuan Lyu, Bharath Irukulapati, and Jaya Prakash Champati. 2026. Speculation at a Distance: Where Edge–Cloud Speculative Decoding Actually Pays Off. arXiv:2606.25091 [cs.DC]

[27] Venkata N. Padmanabhan and Jeffrey C. Mogul. 1996. Using Predictive Prefetching to Improve World Wide Web Latency. In *Proceedings of the ACM Conference on Communications Architectures, Protocols and Applications (SIGCOMM)*. 22–36.

[28] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*.

[29] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. 1995. Informed Prefetching and Caching. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*. 79–95.

[30] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*. 155–170. Best Paper.

[31] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[32] Qwen Team. 2026. Qwen3.8-27B (Hugging Face model card). https://huggingface.co/Qwen/Qwen3.8-27B. Qwen3.8-27B release; W4A16 AutoRound-quantized checkpoint https://huggingface.co/born2bewild/Qwen3.8-27B-W4A16-AutoRound-fast.

[33] Nils Reimers and Tom Gurewitz. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3982–3992.

[34] Stephen Robertson and Stephen Walker. 1994. Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 232–241.

[35] Yifan Shen, Muhammad Umar, Seongmin Maeng, Jaewoong Suh, and Ishan Gupta. 2024. Towards Understanding Systems Tradeoffs in RAG Model Inference. arXiv:2412.11854 [cs.DC]

[36] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 3645–3650.

[37] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. 2017. Distributed Deep Neural Networks over the Cloud, the Edge and End Devices. In *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*. 1–11.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. 5998–6008.

[39] Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2025. Speculative RAG: Enhancing Retrieval-Augmented Generation through Drafting. In *International Conference on Learning Representations (ICLR)*.

[40] Benjamin Wester, Peter M. Chen, James Cowling, Jason Flinn, Edmund B. Nightingale, and Barbara Liskov. 2009. Tolerating Latency in Replicated State Machines through Client Speculation. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[41] Benjamin Wester, Peter M. Chen, and Jason Flinn. 2011. Operating System Support for Application-Specific Speculation. In *Proceedings of the 6th European Conference on Computer Systems (EuroSys)*.

[42] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 193–210.